The following code prints the lines in testfile that have the string on in them:

```
File.foreach("testfile").with_index do |line, index|
  puts "#{index}: #{line}" if line.match?(/on/)
end
```

produces:

```
0: This is line one
3: And so on...
```

You can test to see whether a pattern does not match a string using the negative match operarator !~:

```
File.foreach("testfile").with_index do |line, index|
  puts "#{index}: #{line}" if line !~ /on/
end
```

produces:

```
1: This is line two
2: This is line three
```

Changing Strings with Patterns

The String method sub takes a pattern and some replacement text. (Actually, it does more than that, but we won't get to that for a while.) If it finds a match for the pattern in the string, it replaces the matched substring with the replacement text.

```
str = "Dog and Cat"
new_str = str.sub(/Cat/, "Gerbil")
puts "Let's go to the #{new_str} for a pint."
```

produces:

```
Let's go to the Dog and Gerbil for a pint.
```

The sub method changes only the first match it finds. To replace all matches, use gsub. (The *g* stands for global.)

```
str = "Dog and Cat"
new_str1 = str.sub(/a/, "*")
new_str2 = str.gsub(/a/, "*")
puts "Using sub: #{new_str1}"
puts "Using gsub: #{new_str2}"
```

produces:

```
Using sub: Dog *nd Cat
Using gsub: Dog *nd C*t
```

Both sub and gsub return a new string. (If no substitutions are made, that new string will just be a copy of the original.)

If you want to modify the original string, use the sub! and gsub! forms—often in the Ruby library the use of ! at the end of a method name means the method modifies the receiver in place rather than duplicating it.

```
str = "now is the time"
str.sub!(/i/, "*")
str.gsub!(/t/, "T")
puts str
```